

Student

I understand closures, but decorators still feel like magic. What exactly is a decorator?

ChatGPT • AI Assistant

CONCEPT CARD

A decorator is simply a function that takes another function, modifies or extends its behavior, and returns a new function.

Student

So decorators are really just functions?

ChatGPT • AI Assistant

Exactly. There is no special magic. Decorators are built on top of functions and closures.

Student

Why do developers use decorators so much?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Decorators allow developers to add behavior without modifying the original function. Logging, timing, authentication, caching, validation, monitoring, and access control are common decorator use cases.

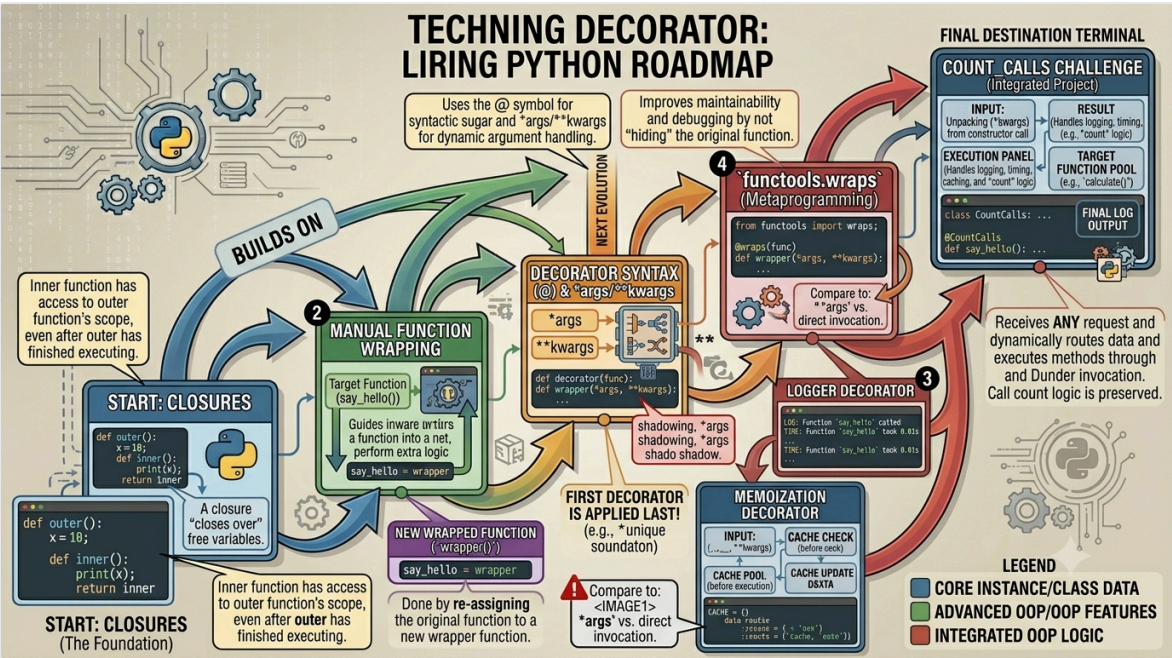
Student

How should I approach this section?

ChatGPT • AI Assistant

WORKBOOK GUIDE

We'll start by manually wrapping functions. Then we'll introduce decorator syntax. Next we'll learn functools.wraps. Finally we'll build three production-style decorators and end with a challenge project.



DECORATOR LEARNING ROADMAP

DECORATOR TOPIC SUMMARY

TOPIC	PURPOSE	REAL WORLD USE	INTERVIEW FREQUENCY
Decorator Basics	Wrap Functions	Code Reuse	Very High
functools.wraps	Preserve Metadata	Professional Code	High
Timer Decorator	Measure Speed	Performance	High
Logger Decorator	Track Calls	Monitoring	High

TOPIC	PURPOSE	REAL WORLD USE	INTERVIEW FREQUENCY
Memoizer	Cache Results	Optimization	Very High

HOW DECORATORS WORK

ChatGPT • AI Assistant

A diagram showing a function entering a decorator, being wrapped by an inner function, and then returning a new decorated function. Arrows highlight closure creation and execution order.

DECORATOR EXECUTION FLOW

Student

Show me decorators from the beginning.

ChatGPT • AI Assistant

CONCEPT CARD

A decorator wraps a function and returns a modified version.

Student

Show me without using @ syntax.

ChatGPT • AI Assistant

EXAMPLE CARD

Step 1 — Original Function

python – Code Example

```
def greet():  
    print('Hello')
```

Step 2 — Wrapper Function

python – Code Example

```
def decorator(func):  
  
    def wrapper():  
        print('Before')  
        func()  
        print('After')  
  
    return wrapper
```

Step 3 — Manual Decoration

python – Code Example

```
greet = decorator(greet)  
  
greet()
```

Output:

text – Code Example

```
Before  
Hello  
After
```

Student

Where is the closure?

ChatGPT • AI Assistant

The wrapper function remembers func even after decorator() finishes.

That remembered variable is the closure.

Student

Why does wrapper still know about func?

ChatGPT • AI Assistant

🔗 ANALOGY

Think of wrapper as carrying a backpack.

The backpack contains func.

Even after decorator() finishes, wrapper still carries that backpack.

Student

How does @ syntax fit in?

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
@decorator
def greet():
    print('Hello')
```

This is equivalent to:

python – Code Example

```
greet = decorator(greet)
```

Student

What's the biggest beginner mistake?

ChatGPT • AI Assistant

❗ COMMON PITFALL / MISTAKE

Forgetting to return wrapper.

Broken Version:

python – Code Example

```
def decorator(func):  
  
    def wrapper():  
        func()
```

Result:

The decorator returns None.

Student

Interview question?

ChatGPT • AI Assistant

🔗 INTERVIEW CARD

Question:

What is a decorator?

Expected Answer:

A function that takes another function, adds behavior, and returns a new function.

Student

Question before continuing?

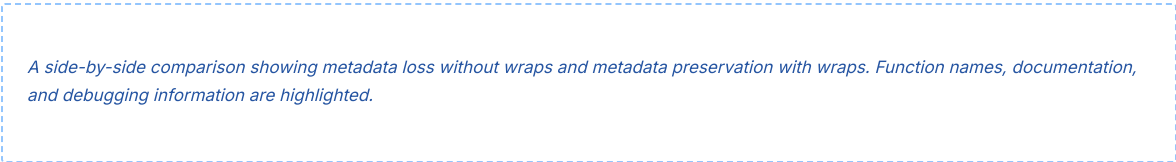
ChatGPT • AI Assistant

[Checkpoint Question]

What object is stored inside the closure of wrapper()?

WHY FUNCTOOLS WRAPS IS NEEDED

ChatGPT • AI Assistant



METADATA PRESERVATION

Student

I've seen functools.wraps everywhere. Why is it necessary?

ChatGPT • AI Assistant

CONCEPT CARD

Decorators accidentally hide metadata from the original function.

Student

What metadata?

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
def greet():  
    pass  
  
print(greet.__name__)
```

Output:

text – Code Example

```
greet
```

Student

What happens after decoration?

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
@decorator
def greet():
    pass

print(greet.__name__)
```

Output:

text – Code Example

```
wrapper
```

Student

That's wrong.

ChatGPT • AI Assistant

Exactly.

The wrapper replaced the original metadata.

Student

How do we fix it?

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
from functools import wraps

def decorator(func):

    @wraps(func)
    def wrapper():
        return func()

    return wrapper
```

Student

What does wraps copy?

ChatGPT • AI Assistant

It preserves:

- **name**
- **doc**
- **module**
- Other metadata

Student

Why does that matter?

ChatGPT • AI Assistant



Debugging tools, documentation systems, frameworks, and IDEs depend on function metadata.

Student

What happens if I forget wraps?

ChatGPT • AI Assistant

❗ **COMMON PITFALL / MISTAKE**

Your decorated functions appear as wrapper instead of their original names.

Student

Interview question?

ChatGPT • AI Assistant

📌 **INTERVIEW CARD**

Question:

Why use functools.wraps?

Expected Answer:

To preserve metadata from the original function.

Student

Question before continuing?

ChatGPT • AI Assistant

[Checkpoint Question]

What would greet.**name** return without wraps?

BUILDING A TIMER DECORATOR

ChatGPT • AI Assistant

A flow diagram showing execution entering the wrapper, recording start time, executing the original function, recording end time, calculating duration, and returning the original result.

TIMER DECORATOR FLOW

Student

Let's build a real decorator.

ChatGPT • AI Assistant

CONCEPT CARD

A timer decorator measures how long a function takes to execute.

Student

Why would I use that?

ChatGPT • AI Assistant

REAL WORLD APPLICATION

Performance optimization.

Developers use timers to identify slow code.

Student

Show me.

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
import time
from functools import wraps

def timer(func):

    @wraps(func)
    def wrapper(*args, **kwargs):

        start = time.time()

        result = func(*args, **kwargs)

        end = time.time()

        print(f'Time: {end-start}')

        return result

    return wrapper
```

Student

Explain the closure.

ChatGPT • AI Assistant

The wrapper remembers func.

Whenever wrapper executes, it calls the stored function and measures execution time.

Student

Show the output.

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
@timer
def slow_task():
    time.sleep(1)

slow_task()
```

Output:

text – Code Example

Time: 1.00

Student

Break it intentionally.

ChatGPT • AI Assistant

[Broken Example]

Remove:

python – Code Example

```
return result
```

Problem:

The decorated function always returns None.

Student

Why is that bad?

ChatGPT • AI Assistant

The original function's result disappears.

Interview question?

ChatGPT • AI Assistant

 INTERVIEW CARD

What happens if a decorator forgets to return the original result?

Student

Question before continuing?

ChatGPT • AI Assistant

[Checkpoint Question]

Why does wrapper need both args and kwargs?

BUILDING A LOGGER DECORATOR

ChatGPT • AI Assistant

A sequence diagram showing logging before execution, function execution, and logging after completion. Callouts explain monitoring and debugging use cases.

LOGGER DECORATOR

Student

What does a logger decorator do?

ChatGPT • AI Assistant

 CONCEPT CARD

It records function activity.

Student

Why is that useful?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Production systems use logging to troubleshoot issues and monitor behavior.

Student

Show me.

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
from functools import wraps

def logger(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')

        result = func(*args, **kwargs)

        print('Finished')

        return result

    return wrapper
```

 Show output.

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
@logger
def greet(name):
    print(name)

greet('Alice')
```

Output:

text – Code Example

```
Calling greet
Alice
Finished
```

 Break it.

ChatGPT • AI Assistant

[Broken Example]

Remove:

python – Code Example

```
func(*args, **kwargs)
```

Result:

The original function never executes.

Student

Interview question?

ChatGPT • AI Assistant

 INTERVIEW CARD

What happens if wrapper never calls func?

Expected Answer:

The original function is skipped entirely.

Student

Question before continuing?

ChatGPT • AI Assistant

[Checkpoint Question]

What line actually executes the original function?

BUILDING A MEMOIZER AND COUNT CALLS CHALLENGE

ChatGPT • AI Assistant

A flowchart showing a function call entering the memoizer. If arguments exist in cache, the cached result is returned immediately. Otherwise the function executes, stores the result in cache, and returns it.

MEMOIZATION CACHE FLOW

Student

What is memoization?

ChatGPT • AI Assistant

CONCEPT CARD

Memoization stores previous results so expensive computations do not run repeatedly.

Student

Show me.

ChatGPT • AI Assistant

EXAMPLE CARD

python — Code Example

```
from functools import wraps

def memoize(func):

    cache = {}

    @wraps(func)
    def wrapper(*args):

        if args in cache:
            return cache[args]

        result = func(*args)

        cache[args] = result

        return result

    return wrapper
```

Student

Where is the closure?

ChatGPT • AI Assistant

The wrapper remembers both func and cache.

Student

Show me the output.

ChatGPT • AI Assistant

EXAMPLE CARD

python – Code Example

```
@memoize
def square(x):
    print('Computing')
    return x**x

square(5)
square(5)
```

Output:

text – Code Example

Computing

The second call uses the cache.

Student

Break it intentionally.

ChatGPT • AI Assistant

[Broken Example]

Remove:

python – Code Example

```
cache[args] = result
```

Result:

Nothing is cached.

Student

Why is memoization useful?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Caching appears in APIs, machine learning, web applications, databases, and scientific computing.

Student

Final exercise?

ChatGPT • AI Assistant

🔥 EXERCISE

Write a decorator named `count_calls`.

Requirements:

1. Count how many times a function executes.
2. Print the count.
3. Preserve metadata using wraps.

Starter:

python — Code Example

```
@count_calls
def greet():
    print('Hello')
```

Give me a hint.

ChatGPT • AI Assistant

[Hint]

Store a counter variable inside the closure.

Student

Show me the solution.

ChatGPT • AI Assistant

EXAMPLE CARD

python — Code Example

```
from functools import wraps

def count_calls(func):

    count = 0

    @wraps(func)
    def wrapper(*args, **kwargs):

        nonlocal count

        count += 1

        print(f'Call #{count}')

        return func(*args, **kwargs)

    return wrapper
```

Student

[Interview question?](#)

ChatGPT • AI Assistant

 INTERVIEW CARD

Explain how closures enable decorators.

Expected Answer:

The inner wrapper remembers variables from the outer decorator function after the outer function finishes execution.

Student

[Final Quiz?](#)

ChatGPT • AI Assistant

 PRACTICE QUIZ

1. What is a decorator?
2. Why is wraps needed?
3. What is a closure?
4. What does a timer decorator measure?
5. What does a logger decorator record?
6. What problem does memoization solve?
7. What variable stores cached results?
8. What keyword allows count to be modified inside count_calls?

Answer Key:

1. A function that wraps another function.
2. Preserve metadata.
3. Remember outer variables.
4. Execution time.
5. Function activity.
6. Repeated computation.

7. cache.

8. nonlocal.

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

SUMMARY CARD

- Decorators wrap functions.
- Closures power decorators.
- wraps preserves metadata.
- Timer decorators measure performance.
- Logger decorators monitor execution.
- Memoizers cache expensive computations.
- count_calls demonstrates closure-based state management.
- Decorators are one of the most common Python interview topics.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including decorators without arguments, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **DECORATOR LEARNING ROADMAP:** A roadmap showing the progression from closures to manual function wrapping, dec...